

Responsible denial of service with web cache poisoning

Responsible denial of service with web cache poisoning - James Kettle, PortSwigger.

- Published: 2019-10-24
- Original: <<https://portswigger.net/research/responsible-denial-of-service-with-web-cache-poisoning>>
- Preserved from: <https://portswigger.net/research/responsible-denial-of-service-with-web-cache-poisoning> (live) on 2026-09-16
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Responsible denial of service with web cache poisoning | PortSwigger Research

Responsible denial of service with web cache poisoning

[image: James Kettle]

James Kettle

Director of Research

[@albinowax](#)

-

****Published: ****Thursday, 24 October 2019 at 12:13 UTC

-

****Updated: ****Thursday, 14 November 2019 at 14:52 UTC

-



In this post, I'll tell the story of how I came to love denial of service attacks, and show you how to use [web cache poisoning](#) to take down websites with a single request while earning bug bounties along the way.

Denial of Service (DoS) attacks have a poor reputation. Historically, DoS used to be trivial - you could knock most sites offline using script-kiddie friendly tools like slowloris.pl. DoS attacks are also often conflated with DDoS attacks, which are near-impossible to truly fix. A line was drawn between a site being 'hacked', and merely being (D)DoS'd. As a result many hackers, myself included, regarded DoS vulnerabilities as lame. Of course 'availability' sits in the CIA triad and therefore CVSS, but that's just another reason why CVSS sucks, right? This was my perspective until a series of events spanning a year changed it.

While I was researching web cache poisoning, I noticed that you could use it for single-request, persistent DoS attacks, but didn't pay much attention until I found I could take out chunks of Tesla's website due to a WAF returning a cacheable "you have been blocked" page whenever it saw the text 'burpcollaborator.net' in a request:

```
`GET /en_GB/roadster?dontpoisoneveryone=1 HTTP/1.1 Host: www.tesla.com Any-Header: burpcollaborator.net`
```

```
HTTP/1.1 403 Forbidden
```

```
Access Denied. Please contact waf@tesla.com`
```

For efficiency reasons the cache key only includes the highlighted values, so anyone who subsequently tried to access that URL would get a cache hit and receive the Access Denied response. The 'dontpoisoneveryone' parameter is crucial to proving that the attack is possible without actually causing site downtime. For an in-depth explanation of this technique, please refer to [Practical Web Cache Poisoning](#).

This was a curious variation on a standard header-based attack, and would make a great example for my presentation, so I tentatively reported it and unexpectedly received a

token \$300 bounty.

Later on, after discovering, reporting, and publishing a critical cache poisoning exploit chain for Drupal, I realised www.hackerone.com was using Drupal all along - but they'd already applied the patch. While attempting to bypass the patch, I noticed the `X-Forwarded-Port` header could be used to persistently [poison a redirect](#) with an invalid port, causing a timeout for everyone trying to access their site:

```
`GET /index.php?dontpoisoneveryone=1 HTTP/1.1 Host: www.hackerone.com X-Forwarded-Port: 123
```

```
HTTP/1.1 302 Found Location: https://www.hackerone.com:123/ `
```

Video tags are not supported by your browser.

I reported this mostly out of annoyance at missing my chance to apply the Drupal exploit, and received a surprising \$2,500 bounty.

Shortly afterward, I was contacted by academic researchers Hoai Viet Nguyen and Luigi Lo lacono who asked my advice on their foray into using web cache poisoning for DoS attacks. I offered some technical advice then politely suggested that denial of service attacks wasn't a topic worthy of serious research, but they wisely ignored me and kept on going.

Then, on Christmas eve, Deliveroo contacted me because someone had taken down their website with cache poisoning and due to some confusion they thought I was responsible.

Finally, while attempting a HTTP Desync attack on a target I can't name, I happened to use Burp Repeater's 'Paste URL as request' function, which creates a HTTP request from a URL and takes its `User-Agent` from the venerable Internet Explorer 9. This accidentally overwrote the cached page with a 'Please update your browser' response. I reported this, and received a eye-opening \$7,500, which was actually more than I got for the desync attack.

At this point I'd earned over \$10,000 by accidentally finding DoS vulnerabilities, and could no longer ignore the niggling idea that maybe some companies do care about DoS. In retrospect, this looks like a solid example of my [research-breadcrumb theory](#) in action. I dusted off [Param Miner](#), tuned the cache-poisoning detection to make it more sensitive if you toggle the 'twitchy cache poison' option, and set out to earn some bounties. Here are a few highlights:

On <https://bitbucket.org>, you could use `X-Forwarded-SSL` header to overwrite certain pages with a response saying 'Contradictory scheme headers', and also use `Transfer-Encoding` to overwrite arbitrary pages. This earned a \$1,800 bounty:

[\[image: image\].png](#))

On <https://paypal.com/>, you could break core functionality by using an invalid `Transfer-Encoding` header to replace crucial JavaScript files from www.paypalobjects.com with the message '501 Not Implemented'. They patched this and [awarded a top-tier \\$9,700 bounty](#).

My colleague Gareth Heyes spotted that you can DoS instagram for people using old browsers or a proxy, by using `Accept-Encoding: br` to force a brotli-encoded response.

`Accept-Encoding` was in the cache key, but using an underscore made the cache miss it. This earned \$500, doubled to \$1,000 as it was donated.

Finally, you can deny access to core Twitter JavaScript files loaded from `abs.twimg.com` and `ton.twimg.com` by using the header `Range: bytes=cow` to cause a 400 response. They decided not to patch or pay out for this, so you can (carefully) use it for practise if you want.

Other successful attacks used the `Accept` , `Upgrade` , `[Origin]` (<https://nathandavison.com/blog/corsing-a-denial-of-service-via-cache-poisoning>) , and `Max-Forwards` headers.

Most bug-bounty policies have text discouraging DoS attacks. However, if you look closely you'll see some of them actually forbid launching DoS attacks, rather than forbidding reporting DoS vulnerabilities. Web cache poisoning has a rare property in that it's often possible to make a proof of concept without actually launching an attack, provided you use a cache-buster.

That said, quite a few programs do have a blanket exclusion on reporting DoS vulnerabilities. I'd suggest that sufficiently mature programs think about their business requirements, and perhaps tweak their wording to merely exclude 'volumetric DoS' or 'computational DoS' reports.

Finally, the academic research mentioned earlier was actually published earlier this week - you can read it at <https://cpdos.org/>

[web cache poisoning](#)

[Back to all articles](#)